

Chương 6: Cây Quyết định (Decision Trees)

Giảng viên

Đại học Đông Á

August 11, 2026

Mục tiêu của Chương

- Hiểu rõ thuật toán **Cây Quyết định (Decision Trees)** và khả năng của nó.
- Cách một mô hình cây đưa ra dự đoán (Phân loại và Hồi quy).
- Thuật toán **CART** (Classification and Regression Tree) dùng để huấn luyện.
- Tìm hiểu về độ vẩn đục: **Gini Impurity** và **Entropy**.
- Các kỹ thuật **Chính quy hóa (Regularization)** bằng siêu tham số.
- Điểm yếu của Cây Quyết định: Phương sai cao (High Variance) và nhạy cảm với hướng trục.

Mục lục

1. Khái quát về Cây Quyết định
2. Huấn luyện và Trục quan hóa
3. Đưa ra dự đoán (Making Predictions)
4. Thuật toán Huấn luyện CART
5. Chính quy hóa Cây (Regularization)
6. Hồi quy bằng Cây quyết định
7. Các Điểm yếu của Cây Quyết Định
8. Tổng kết

1.1 Sức mạnh của Decision Trees

- Cây Quyết định là một trong những thuật toán Học máy đa năng nhất.
- Có thể thực hiện cả **Phân loại (Classification)**, **Hồi quy (Regression)**, và học nhiều đầu ra (Multioutput).
- Khả năng khớp với các tập dữ liệu cực kỳ phức tạp (Non-linear datasets).
- Là thành phần cơ bản (Building block) của các thuật toán cực mạnh như **Random Forest** và **Gradient Boosting**.

1.2 Mô hình Hộp Trắng (White Box Model)

- Điểm sáng giá nhất của Cây Quyết định là tính **dễ diễn giải (Interpretability)**.
- Nó được gọi là mô hình Hộp Trắng (White Box) vì ta có thể theo dõi chính xác cách mô hình đi đến kết luận.
- Trái ngược với mô hình Hộp Đen (Black Box) như Mạng nơ-ron hay Random Forest, nơi rất khó giải thích tại sao mô hình lại đưa ra dự đoán đó.
- Được sử dụng rộng rãi trong các lĩnh vực Y tế, Tài chính, nơi cần giải trình quyết định.

2.1 Code: Huấn luyện cây quyết định

- Sử dụng bộ dữ liệu kinh điển Iris (Hoa diên vĩ).
- Ta sẽ chỉ sử dụng 2 đặc trưng: petal length (chiều dài cánh hoa) và petal width (chiều rộng).

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

# Tải tập dữ liệu Iris
iris = load_iris(as_frame=True)
X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
y_iris = iris.target

# Huấn luyện mô hình Cây Quyết định
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X_iris, y_iris)
```

2.2 Code: Xuất biểu đồ cây (Graphviz)

- Scikit-Learn cung cấp phương thức `export_graphviz` để vẽ cây thành file `.dot`.
- Sau đó dùng Graphviz để chuyển `.dot` thành ảnh PNG hoặc PDF.

```
from sklearn.tree import export_graphviz

export_graphviz(
    tree_clf,
    out_file="iris_tree.dot",
    feature_names=["petal length (cm)", "petal width (cm)"],
    class_names=iris.target_names,
    rounded=True,
    filled=True
)

# Lệnh terminal biến dịch dot sang png:
# dot -Tpng iris_tree.dot -o iris_tree.png
```

2.3 Trực quan hóa Cây Iris

- Nút trên cùng gọi là **Nút gốc (Root Node)**.
- Nút không có con gọi là **Nút lá (Leaf Node)**.
- Mỗi nút hiển thị: Điều kiện chia, số samples (mẫu), value (phân bố), và class (nhãn dự đoán).

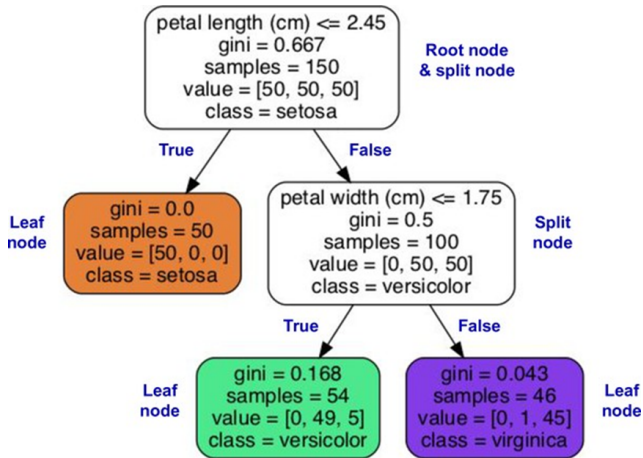


Figure: Hình 6-1: Cây Quyết định bộ dữ liệu Iris

3.1 Quá trình đưa ra dự đoán

- Cây sẽ duyệt từ **Nút gốc** đi xuống.
- Ở mỗi nút, nó kiểm tra một điều kiện (VD: $\text{petal_length} \leq 2.45 \text{ cm}$).
- Nếu **Đúng (True)**, đi sang nhánh bên Trái.
- Nếu **Sai (False)**, đi sang nhánh bên Phải.
- Lặp lại đến khi gặp Nút lá. Lớp có số lượng mẫu đông nhất trong nút lá đó sẽ là dự đoán cuối cùng.

3.2 Hiểu về Gini Impurity (Độ vẩn đục Gini)

- Tham số gini ở mỗi nút đo lường **độ vẩn đục** (impurity) của nó.
- Một nút được gọi là "thuần khiết" (pure) nếu $gini = 0$, nghĩa là mọi mẫu trong nút đó đều thuộc về **cùng một lớp**.
- Công thức Gini Impurity cho nút i :

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

- Trong đó: $p_{i,k}$ là tỷ lệ phần trăm mẫu thuộc lớp k nằm trong nút i .

3.3 Ví dụ tính toán Gini Impurity

- Xem lại Hình 6-1, Nút màu xanh lục (phía dưới bên trái):
- Có tổng cộng 54 mẫu. Lớp Setosa (0), Lớp Versicolor (49), Lớp Virginica (5).
- Tỷ lệ p : $p_{versicolor} = 49/54$, $p_{virginica} = 5/54$.
- Áp dụng công thức:

$$G_i = 1 - \left(\left(\frac{0}{54} \right)^2 + \left(\frac{49}{54} \right)^2 + \left(\frac{5}{54} \right)^2 \right)$$

$$G_i \approx 1 - (0 + 0.823 + 0.008) \approx 0.168$$

3.4 Ranh giới Quyết định (Decision Boundaries)

- Hình ảnh minh họa không gian đặc trưng.
- Đường nét đậm ở 2.45 cm (Trục tung) là ranh giới từ nút gốc.
- Đường đứt nét ở 1.75 cm (Trục hoành) là ranh giới chia nhánh thứ hai.
- Cây luôn tạo ra các đường ranh giới **vuông góc với trục tọa độ**.

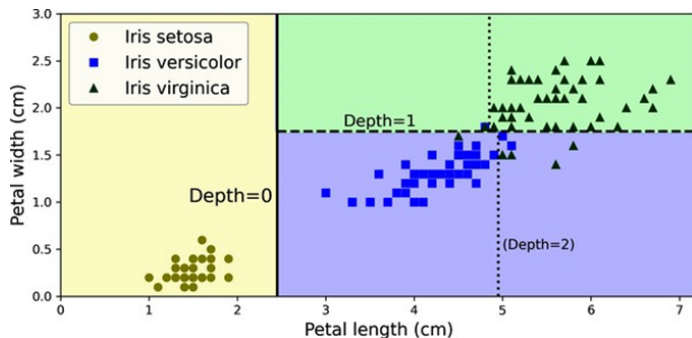


Figure: Hình 6-2: Ranh giới quyết định của Cây

3.5 Không cần Chuẩn hóa (Scaling) dữ liệu!

- Một ưu điểm tuyệt vời khác của Cây Quyết định: **Nó gần như không yêu cầu tiền xử lý dữ liệu!**
- Không cần StandardScaler hay MinMaxScaler.
- Việc một đặc trưng chạy từ 0-1 còn đặc trưng khác chạy từ 0-1000 không hề ảnh hưởng đến việc phân chia nhánh. Thuật toán chỉ tìm điểm ngưỡng cắt t_k .

3.6 Ước tính Xác suất Lớp (Class Probabilities)

- Cây quyết định cũng có thể trả về **xác suất** mà một mẫu thuộc về một lớp cụ thể.
- Bằng cách duyệt từ gốc xuống một nút lá.
- Xác suất của lớp k chính là tỉ lệ mẫu của lớp k trên tổng số mẫu tại nút lá đó.
- VD: Bông hoa dài 5cm, rộng 1.5cm sẽ đi vào nút lá màu xanh lục (49 Versicolor, 5 Virginica). Xác suất là $49/54 \approx 90.7\%$ cho Versicolor.

3.7 Code: Dự đoán Xác suất

```
# Dự đoán xác suất cho 1 bông hoa:
# petal_length = 5.0, petal_width = 1.5
probabilities = tree_clf.predict_proba([[5, 1.5]])
print("Xác suất (Setosa, Versi, Virgi):", probabilities)
# Output: [[0.          0.90740741 0.09259259]]

# Dự đoán lớp (argmax của xác suất)
pred_class = tree_clf.predict([[5, 1.5]])
print("Lớp dự đoán:", pred_class)
# Output: [1] (tức là Versicolor)
```